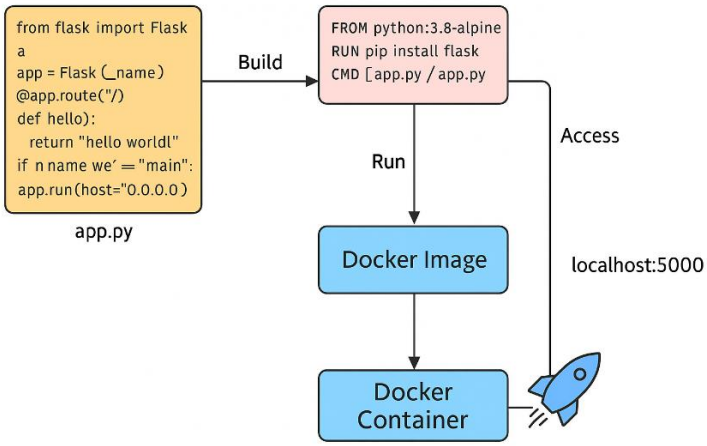
Conteneuriser cette application avec **Podman**

- application web avec Python Flask -

🎯 Objectifs :

- Créer une **petite application web avec Python Flask** (elle renvoie juste "hello world").
- Conteneuriser cette application avec **Podman** (comme on le ferait avec Docker).
- **Tester le conteneur** pour s'assurer que l'application fonctionne bien dans un environnement isolé.



◇ 1. Installer Flask et créer l'application

a) dnf install python3-flask

```
 * Requesting data...
 * Testing changes...
 * Installing packages...
pip 21.3.1 from /usr/lib/python3.9/site-packages/pip (python 3.9)

[ons@mylocalhost ~]$
```

```
Collecting zipp>=3.20
  Downloading zipp-3.23.1-py3-none-any.whl (10 kB)
Installing collected packages: zipp, markupsafe, werkzeug, jinja2, itsdangerous,
 importlib-metadata, click, blinker, flask
Successfully installed blinker-1.9.0 click-8.1.8 flask-3.1.3 importlib-metadata-
8.7.1 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 werkzeug-3.1.8 zipp-3.23.
1
```

```
[ons@mylocalhost ~]$ python3 -c "import flask; print(flask.__version__)"
<string>:1: DeprecationWarning: The '__version__' attribute is deprecated and wi
ll be removed in Flask 3.2. Use feature detection or 'importlib.metadata.version
("flask")' instead.
3.1.3
[ons@mylocalhost ~]$
```

- 🧠 **Flask** est un mini-framework Python pour créer facilement des sites web.
- Cette commande installe Flask via DNF (le gestionnaire de paquets sur Fedora/RedHat).

b) Créer un fichier `app.py` :

```
[ons@mylocalhost ~]$ nano app.py
```

Voici ce qu'il fait :

```
from flask import Flask

app = Flask(__name__)  # Création de l'application

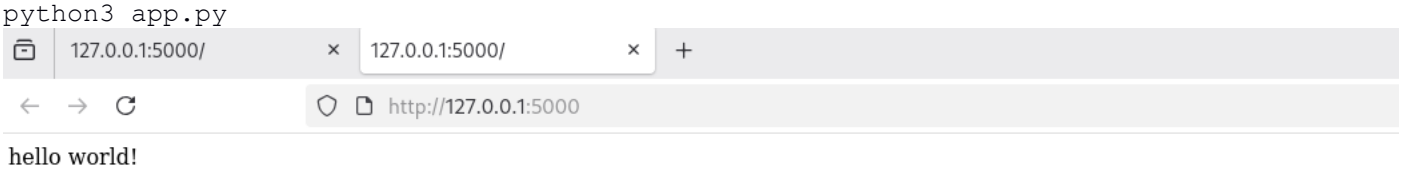
@app.route("/")        # Route racine
def hello():
    return "hello world!"  # Ce qui s'affiche dans le navigateur

if __name__ == "__main__":
    app.run(host="0.0.0.0")  # Lance le serveur web sur toutes les IP de la machine
```

🧠 Résultat : Une fois lancé, si tu vas sur `http://localhost:5000`, tu verras : `hello world!`

◇ 2. Lancer l'application :

```
python3 app.py
```



- Cela **démarre le serveur web flask** sur le port **5000**.
- Tu testes si l'appli fonctionne **avant de la conteneuriser**.

◇ 3. Créer un Dockerfile (ou Podmanfile)

Voici ce que fait chaque ligne :

```
FROM python:3.8-alpine       # Utilise une image légère de Python
RUN pip install flask        # Installe flask dans le conteneur
COPY app.py /app.py          # Copie ton code Python dans l'image
CMD ["python", "app.py"]     # Commande qui démarre l'application
```

```
FROM python:3.8-alpine

RUN pip install flask

COPY app.py /app.py

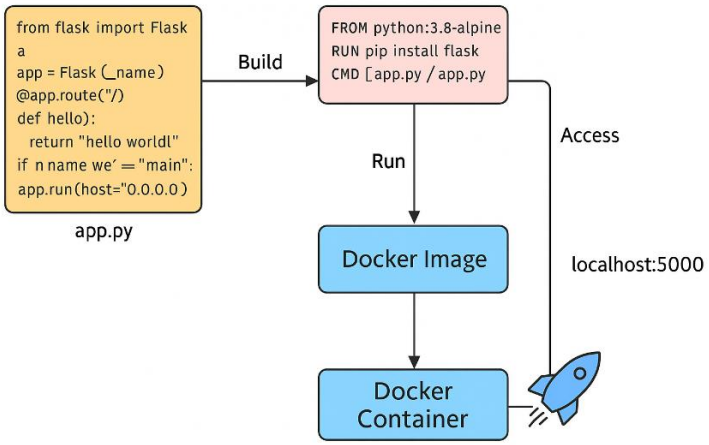
CMD ["python", "app.py"]
```

🤓 C'est comme une **recette de cuisine** pour construire ton environnement contenant Python + Flask + ton code.

◇ 4. Construire l'image :

```
podman build -t python-hello-world .
STEP 4/4: CMD ["python", "app.py"]
COMMIT python-hello-world
--> 3039966c65d9
Successfully tagged localhost/python-hello-world:latest
3039966c65d9c3ff4d0de83a7774c8b55b618b087e12669775168e67e3a79d0e
[ons@mylocalhost ~]$
```

- Cela **crée une image** nommée `python-hello-world` à partir du Dockerfile.
- Le `.` signifie "dans le dossier courant" (où il y a le Dockerfile et `app.py`).



◇ 5. Taguer l'image :

```
podman tag python-hello-world python-hello-world:tekup
[ons@mylocalhost ~]$ podman tag python-hello-world python-hello-world:tekup
[ons@mylocalhost ~]$
```

- On crée un **alias (ou une étiquette)** de cette image avec le tag `tekup`.
- Cela peut être utile pour **organiser ou versionner** les images.

◇ 6. Lancer un conteneur :

```
podman run -dit --name container-hello -p 5000:5000 python-hello-world
[ons@mylocalhost ~]$ podman run -dit --name container-hello -p 5000:5000 python-
hello-world
b564a903839a3d91672d6c3eeec92d2bfdade3486ca642d987d85af07984784b
[ons@mylocalhost ~]$
```

📌 Détail :

- `-d` : mode détaché (en arrière-plan)
- `-i` : mode interactif
- `-t` : attache un terminal
- `--name` : nom du conteneur
- `-p 5000:5000` : redirige le port de l'hôte vers celui du conteneur

🧠 Le conteneur va **écouter sur le port 5000** → accessible via `http://localhost:5000`

---

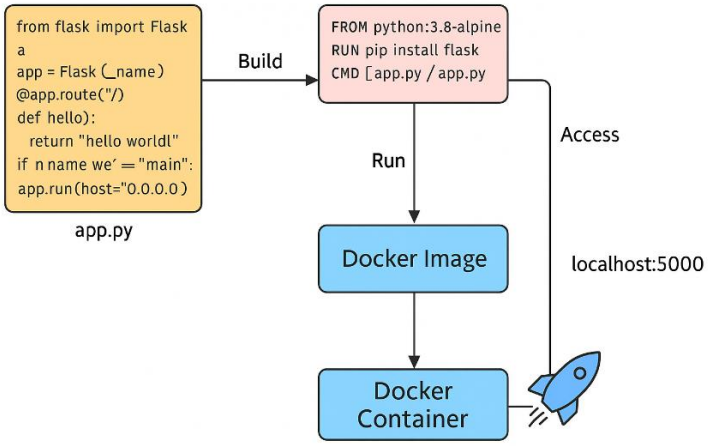
7. Tester dans le navigateur ou avec curl :

```
curl http://localhost:5000
```

```
[ons@mylocalhost ~]$ curl http://localhost:5000
hello world![ons@mylocalhost ~]$
```

☑ Tu dois voir :

```
hello world!
```



☑ Résultat final :

Tu as :

- Une application Python simple.
- Une image conteneurisée qui contient ton app.
- Un conteneur lancé depuis cette image.
- Et un accès à ton site via un navigateur ou curl.